

Program No: 8

File Name:

Ex. No:

Date: _____

IMPLEMENT AUTOMATED TESTING FOR A JAVASCRIPT-BASED HTML APPLICATION

AIM:

To learn how to write and run automated unit tests for JavaScript functions used in a web application using Jest testing framework. This helps ensure code correctness and detect bugs early.

ALGORITHM:

1. Create your JavaScript functions file, for example `app.js`, which contains the code to be tested.
2. Export the functions in `app.js` using `module.exports` to make them accessible for testing.
3. Initialize a Node.js project in VS Code by running `npm init -y` in the terminal.
4. Install Jest as a development dependency with `npm install --save-dev jest`.
5. Add a test script in `package.json` under `scripts`: `"test": "jest"`.
6. Create a test file, e.g. `app.test.js`, where you import the functions from `app.js`.
7. Write test cases using Jest's `test()` and `expect()` functions to check expected output.
8. Run automated tests in the terminal by running `npm test`.
9. Optionally, use the Live Server extension in VS Code to serve and view the HTML page during development.

PROGRAM:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <title>Tested App</title>

  <script src="app.js" defer></script>

</head>

<body>

  <h1>Simple App for Testing</h1>

</body>

</html>

function add(a, b) {

  return a + b;

}

function multiply(a, b) {

  return a * b;

}

// Export functions for Jest
```

```
if (typeof module !== 'undefined') {  
  
  module.exports = { add, multiply };  
  
}
```

```
npm init -y
```

```
npm install --save-dev jest
```

```
"scripts": {  
  
  "test": "jest"  
  
}
```

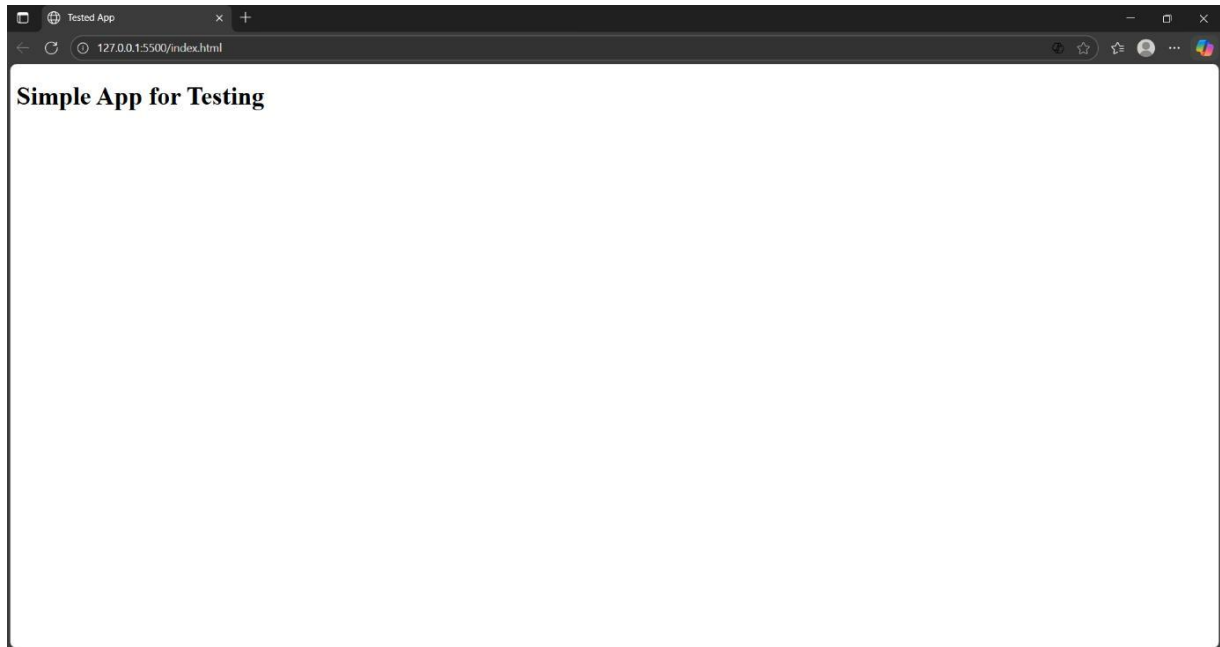
```
const { add, multiply } = require('./app.js');
```

```
test('adds two numbers', () => {  
  
  expect(add(2, 3)).toBe(5);  
  
});
```

```
test('multiplies two numbers', () => {  
  
  expect(multiply(2, 3)).toBe(6);  
  
});
```

```
npm test
```

OUTPUT:



RESULT:

Thus the program was successfully executed.